DuocUC | ONLINE



Guía de aprendizaje

Configuración de jobs y steps en Spring Batch

Exp 1 – Semana 2

Desarrollo Backend III (PBY2203)

Facilitador disciplinar: Ana Karina Villagrán Ibarra

Asesor par: Ignacio Villaroel

Índice

Introducción a la semana....3
Resultado de aprendizaje....4
Conceptos relevantes....4
Preguntas activadoras....5
Actividad....5
Configuración de jobs y steps en Spring Batch....6
Escalado y procesamiento paralelo....6
Políticas de finalización y re-ejecución....9
Optimización de recursos del sistema....9
Manejo de errores y excepciones....10
Caso práctico: “FreshMart”....11
Estructura general del proyecto....12
Manejo de errores....20
Resultado final....21
Cierre de la semana....22
Referencias....23
Lecturas de la semana....23
Apuntes....24

Introducción a la semana



Ya te encuentras en la semana 2 llamada Configuración de jobs y steps en Spring Batch, cuyo logro será la implementación de escalado, procesamiento paralelo y diversas políticas de jobs en procesos batch para optimizar tiempos de ejecución, estabilidad y continuidad del sistema.

Hasta ahora, has aprendido los fundamentos de Spring Batch y cómo estructurar trabajos por lotes para la migración de sistemas legacy, centrándote en la lectura, transformación y escritura de datos. Con este nuevo desafío, profundizarás en la optimización de procesos batch al implementar técnicas de escalado y procesamiento paralelo. Desarrollarás habilidades para configurar políticas que mejoren la estabilidad y la resiliencia del sistema, lo que te permitirá manejar grandes volúmenes de datos de manera más eficiente y recuperarte ante errores de forma más rápida.

En esta guía de aprendizaje encontrarás:

- ¿Cómo configurar y optimizar jobs y steps en Spring Batch para manejar eficientemente grandes volúmenes de datos?
- ¿Qué técnicas de escalado y procesamiento paralelo se pueden implementar para mejorar los tiempos de ejecución?
- ¿Cómo configurar políticas de finalización y reintento para garantizar la resiliencia del sistema en caso de errores o fallos?
-

Resultado de aprendizaje

El estudiante será capaz de:

RA1. Desarrolla componentes batch para la migración de sistemas antiguos basados en cobol/shell.

Indicador de logro:

IL3. Implementa escalado, procesamiento paralelo y diversas políticas de jobs en procesos batch para optimizar tiempos de ejecución, estabilidad y continuidad del sistema.

Conceptos relevantes

Procesamiento paralelo	Chunks	Políticas personalizadas
Optimización de recursos	Errores y Excepciones	Threads/Hilos

Preguntas activadoras

- ¿Por qué crees que es importante dividir un proceso batch en múltiples pasos (steps)?
- ¿Cómo imaginas que el procesamiento paralelo podría mejorar la eficiencia de un sistema batch?
- ¿Has trabajado anteriormente con procesos que requieren manejar errores o fallos?
- ¿Qué técnicas usaste para asegurar que el proceso continuara sin interrumpirse?

Actividad

Descripción de la actividad

En esta segunda semana realizarás la actividad formativa grupal, llamada "Configurando jobs y steps en Spring Batch" donde tendrás que -como su nombre lo indica- configurar jobs y steps dentro de Spring Batch, optimizando los procesos batch para maximizar su eficiencia y asegurar su correcto funcionamiento bajo diferentes condiciones. Deberás implementar técnicas de escalado y procesamiento paralelo para manejar grandes volúmenes de datos, mejorando los tiempos de ejecución de los jobs. Además, tendrás que crear políticas de finalización, repetición y reejecución de jobs, lo que permitirá que el sistema batch sea más resiliente y capaz de recuperarse ante fallos o errores durante su ejecución.

Configuración de jobs y steps en Spring Batch

En Spring Batch, tienes a tu disposición varias configuraciones clave que te permiten manejar eficientemente la carga de trabajo y optimizar los recursos en el procesamiento de datos. Estas configuraciones, como el **procesamiento en chunks,** las **políticas de reintento** y el **manejo de excepciones,** cumplen un rol fundamental en cómo gestionas los datos y en la capacidad de tu sistema para responder ante errores, ya sean temporales o permanentes.

Por ejemplo, el procesamiento en chunks te facilita dividir grandes volúmenes de datos en bloques más manejables, que se procesan y confirman como una unidad. Esto no solo reduce el consumo de memoria, sino que también asegura que, en caso de fallos, puedas mantener la consistencia y la integridad del proceso. Estas técnicas de configuración son fundamentales para que construyas un sistema batch robusto, flexible y capaz de adaptarse a distintos niveles de carga y posibles fallos.

A continuación, comenzaremos con la revisión del escalado y el procesamiento paralelo en Spring Batch.

Escalado y procesamiento paralelo

El escalado y el procesamiento paralelo en Spring Batch son técnicas clave para optimizar la ejecución de procesos batch al distribuir la carga de trabajo entre múltiples hilos o nodos, lo que mejora significativamente los tiempos de ejecución y la eficiencia del sistema. Estas estrategias permiten dividir el procesamiento en partes más pequeñas que se ejecutan simultáneamente, ya sea mediante chunks, paralelización de hilos o ejecución distribuida, garantizando un uso óptimo de los recursos y la capacidad de manejar grandes volúmenes de datos de manera efectiva.

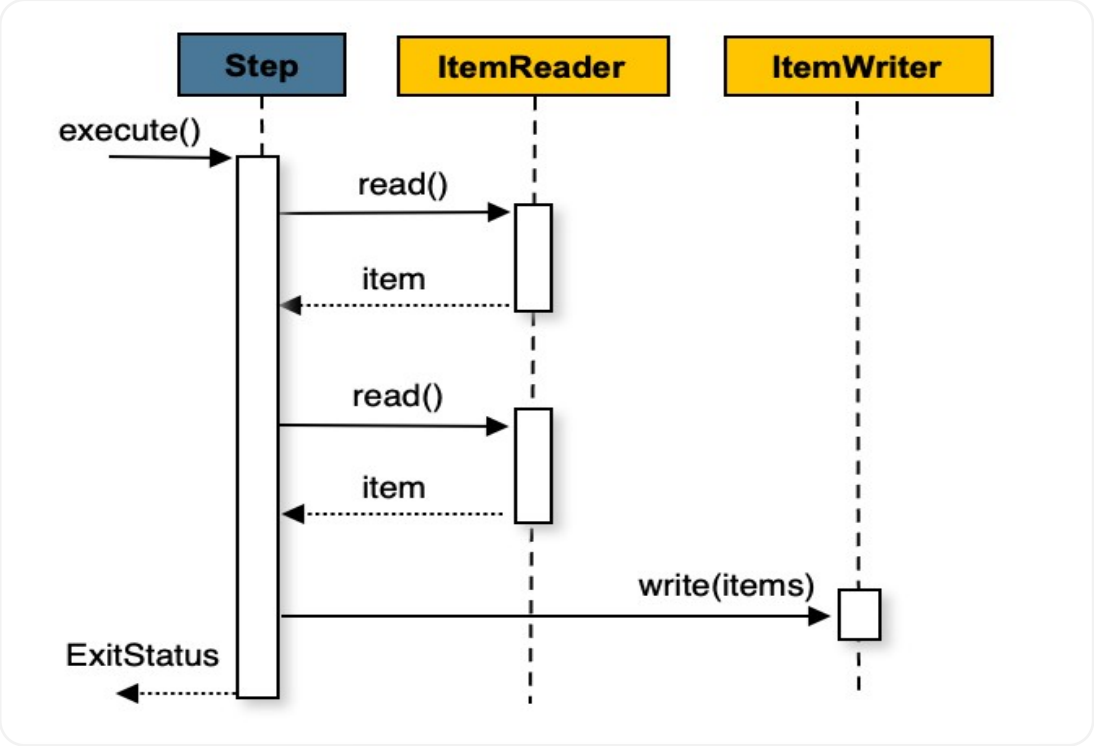
Técnicas de escalado

Procesamiento en chunks

El procesamiento en chunks en Spring Batch te permite dividir un conjunto de datos en fragmentos más pequeños para ser procesados de manera eficiente. Cada chunk consiste en un grupo de registros que tú lees, procesas y escribes juntos, lo que facilita la administración de la memoria y mejora el rendimiento. Al trabajar con lotes más pequeños, optimizas el uso de los recursos del sistema y puedes controlar mejor el manejo de errores, ya que cualquier fallo en el procesamiento de un chunk específico no afecta a todo el conjunto de datos, permitiéndote reintentar o saltar solo los registros problemáticos.

Figura 1

Procesamiento por Chunks



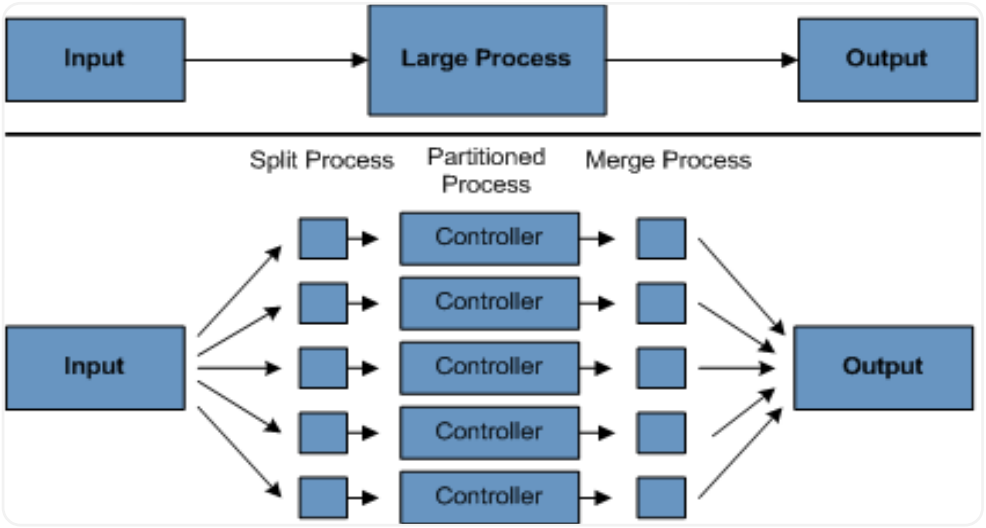
Nota. Flujo de un Step en Spring Batch, donde el ItemReader lee datos (como chunks) y el ItemWriter los escribe. *SpringSource (s.f.).* Chunk-oriented Processing. https://docs.spring.io/spring-batch/reference/step/chunk-oriented-processing.html

Paralelismo con múltiples hilos (multithreading)

El paralelismo con múltiples hilos (multithreading) en Spring Batch te permite ejecutar diferentes partes de un proceso batch de manera simultánea, utilizando varios hilos para distribuir la carga de trabajo. Al implementar esta técnica, optimizas el rendimiento aprovechando mejor los recursos del sistema y reduciendo los tiempos de ejecución al procesar múltiples tareas en paralelo. Puedes configurar Spring Batch para que varios steps se ejecuten al mismo tiempo o para que un mismo step procese diferentes chunks en hilos separados, lo cual es especialmente útil cuando necesitas manejar grandes volúmenes de datos de forma más rápida y eficiente.

Figura 2

Componentes básicos de un proceso



Nota. Diagrama muestra cómo dividir un proceso grande en partes más pequeñas, optimizando el rendimiento al distribuir la carga. *SpringSource (s.f.).* Spring Batch Architecture. https://docs.spring.io/spring-batch/reference/spring-batch-architecture.html

Políticas de finalización y re-ejecución

Las políticas de finalización y re-ejecución en Spring Batch te permiten controlar cómo se manejan los trabajos al completarse o fallar. Configura las condiciones para determinar cuándo un job debe finalizar, reintentarse o reiniciarse, garantizando así la estabilidad del sistema y la continuidad del procesamiento de datos.

Las políticas disponibles en Spring batch son:

- Políticas de omisión (SkipPolicy)
- Políticas de reintento (RetryPolicy)
- Control de finalización (JobExecutionDecider)
- Uso del JobRepository para persistir el estado de ejecución y permitir la reanudación de un job en caso de fallo.

Optimización de recursos del sistema

En el contexto profesional, es común que los recursos sean limitados, por lo que es fundamental aprender técnicas de optimización. Ajustar la configuración para maximizar el rendimiento y minimizar el consumo de memoria es clave. Al configurar adecuadamente el tamaño de los chunks, los hilos y la memoria, podrás procesar grandes volúmenes de datos de manera eficiente y estable.

Estrategias de optimización que deberás considerar en cualquier proyecto batch:

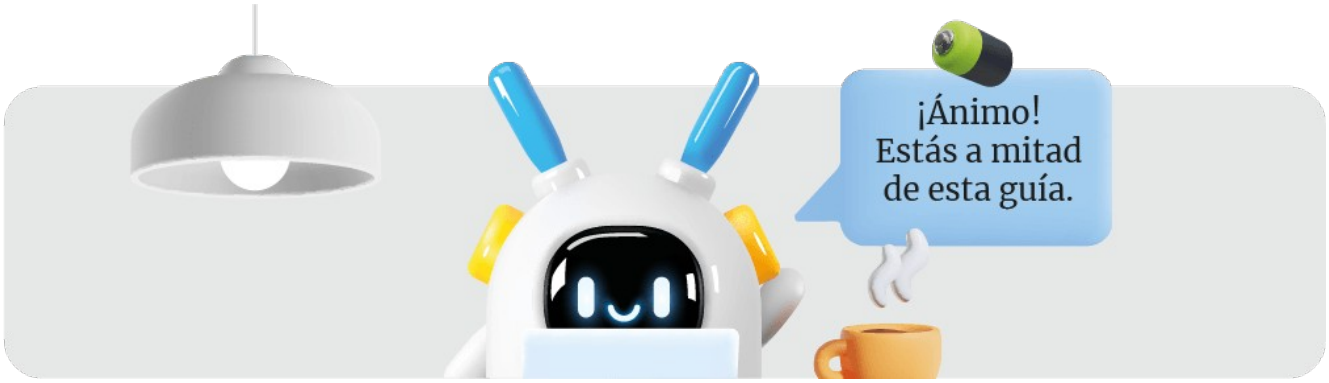
- Configurar parámetros para limitar el uso de memoria durante el procesamiento batch, evitando problemas de rendimiento.
- Configurar el tamaño de los chunks y el número de threads para maximizar la eficiencia sin saturar los recursos del sistema.
- Implementar mecanismos para monitorear el rendimiento de los jobs y ajustar los parámetros en tiempo real según sea necesario.

Manejo de errores y excepciones

Al manejar errores y excepciones en Spring Batch, configuras políticas que te permiten controlar cómo se gestionan los fallos durante la ejecución. Puedes definir reintentos, omisiones o acciones personalizadas para casos específicos, asegurando que los errores no detengan el procesamiento completo y manteniendo la estabilidad del sistema.

Estrategias útiles para el manejo de errores:

- Configuración de listeners y manejadores de errores (StepListener, JobListener).
- Uso de listeners para realizar acciones específicas antes, después o en caso de error en un step o job.
- Registro de logs y seguimiento de ejecuciones.
- Implementación de mecanismos para registrar errores y analizar fallos en la ejecución batch.



Caso práctico: “FreshMart”

Para poder interiorizar en los conceptos anteriores, revisa la siguiente implementación real, y con ello, comprender como se pueden configurar procesos batch.

Contexto

La cadena de supermercados FreshMart ha experimentado un crecimiento explosivo durante los últimos años, expandiéndose rápidamente a nuevas ciudades y abriendo sucursales en diversas zonas. Cada día, millones de transacciones de ventas, registros de inventario y datos de proveedores son generados por las sucursales, pero el sistema actual de procesamiento de datos sigue utilizando tecnología obsoleta basada en scripts batch programados manualmente. Esto ha resultado en problemas de lentitud, errores en la consolidación de datos y una incapacidad para reaccionar rápidamente a cambios en la demanda.

Para mejorar la eficiencia operativa y aprovechar al máximo los datos generados, FreshMart ha decidido modernizar su sistema de procesamiento batch utilizando Spring Batch. El objetivo es implementar un sistema que permita manejar grandes volúmenes de datos de ventas e inventario de manera rápida y eficiente, asegurando que las decisiones de negocio se basen en información precisa y actualizada.

Requerimiento del proyecto

El nuevo sistema batch deberá gestionar un proceso clave:

- **Consolidación diaria de ventas:** cada noche, los datos de ventas de todas las sucursales deben ser procesados para generar un informe consolidado que muestre las tendencias de compra, productos más vendidos y aquellos que han tenido baja rotación. Los datos se reciben de diferentes sistemas de punto de venta, por lo que es esencial procesarlos en paralelo para consolidar la información rápidamente.

Según el caso FreshMart, se desarrollará la solución considerando estrategias de optimización, manejo de errores y escalado.

Links de interés

Accede al siguiente enlace para poder comprender desde una implementación cómo se podría resolver el problema de FreshMart. https://ava.duoc.cl/bbcswebdav/xid-5043431_1

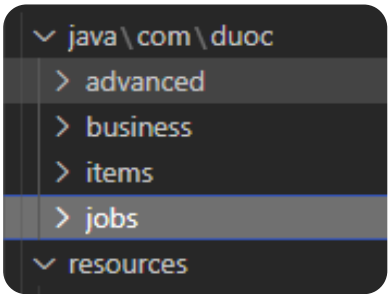
A continuación, podrás repasar las partes importantes de la solución implementada:

Estructura general del proyecto

Dado que ya tienes un dominio profundo sobre los packages en Java, organizar tu solución será clave al implementar un buen proyecto con Spring Batch. Para la implementación de FreshMart, se decidió crear cuatro paquetes, como se muestra en la Figura 3, con el fin de lograr una organización lógica de los recursos.

Figura 3

Componentes básicos de un proceso



Nota. Captura de pantalla del explorador de archivo, que muestra la estructura general de los paquetes de java en la solución. *Microsoft. (2024).* Visual Studio Code. (1.95). [Software]. https://code.visualstudio.com/

Configuración principal del batch

Como puedes ver en la siguiente imagen, la clase ‘VentasJobConfig’ configura un trabajo batch en Spring con un Job que incluye un paso (Step) y un decider (CustomDecider) para manejar el flujo de ejecución. Utilizando anotaciones de Spring, organiza la configuración y escaneo de componentes. El método ‘ventasJob’ crea el trabajo con un identificador incremental y agrega un listener para gestionar eventos del trabajo. El flujo comienza con el paso inicial y luego, según el resultado del decider, continúa o reinicia el paso si el estado es "RETRY". Este método garantiza una estructura clara y controlada del flujo del trabajo batch.

Figura 4

Configuración principal del job

```java
@Configuration // Indica que esta clase contiene la configuración de beans de Spring
@EnableBatchProcessing // Habilita el procesamiento batch en el contexto de Spring
@Import(DataSourceConfiguration.class) // Importa la configuración de la base de datos
@ComponentScans({
    @ComponentScan(basePackages = "com.duoc.items"),
    @ComponentScan(basePackages = "com.duoc.jobs")
})
public class VentasJobConfig {

    // Configuración del Job con ID incremental y Decisor para manejo de finalización
    @Bean
    public Job ventasJob(JobRepository jobRepository, Step step, CustomDecider decider, JobCompletionListener listener) {
        return new JobBuilder(name:"ventasJob", jobRepository) // Crea un Job con el nombre "ventasJob" usando el JobRepository
                .incrementer(new RunIdIncrementer()) // Agrega un incrementador de ID de ejecución
                .listener(listener) // Registra un listener para el Job
                .start(step) // Establece el Step inicial para el Job
                .next(decider) // Usa un Decider para determinar el flujo del Job
                .on(pattern:"COMPLETED").end() // Si el Decider devuelve "COMPLETED", termina el Job
                .from(decider).on(pattern:"RETRY").to(step) // Si el Decider devuelve "RETRY", vuelve a ejecutar el Step
                .end() // Finaliza la configuración del flujo
                .build(); // Construye y devuelve el Job configurado
    }
```

Nota. Configuración principal del job con ID incremental y decisor para manejo de finalización a través de Power Shell. *Microsoft (2006).* Power Shell (7.4.5). [Software]. https://learn.microsoft.com/en-us/powershell/

En la Figura 5 se configura un Step en un trabajo batch de Spring, que realiza el procesamiento paralelo de datos con tolerancia a fallos y manejo de errores. El StepBuilder define el tamaño de procesamiento en chunks de 2 elementos, aplicando una política de transacción. La tolerancia a fallos se habilita mediante faultTolerant(), lo cual permite que el paso maneje errores sin detenerse.

La política de manejo de errores personalizada, establecida con .skipPolicy(fileVerificationSkipper), define qué excepciones pueden ignorarse y cuántas veces. Además, se agregan SkipListener y StepExecutionListener para gestionar errores específicos y ejecutar acciones adicionales cuando ocurren omisiones o se completa un paso.

Figura 5

Componentes avanzados de un step

```
// Step configurado para procesamiento paralelo, reintentos y política de repetición
@Bean
public Step consolidacionDiariaStep(JobRepository jobRepository,
                                    JdbcTransactionManager transactionManager,
                                    VentasItemReader itemReader,
                                    VentasItemProcessor itemProcessor,
                                    FlatFileItemWriter<InformeVenta> itemWriter,
                                    ErrorItemWriter errorItemWriter,
                                    FileVerificationSkipper fileVerificationSkipper,
                                    SkipListener<Venta, InformeVenta> skipListener,
                                    StepExecutionListener errorFileStepExecutionListener,
                                    ThreadPoolTaskExecutor taskExecutor
                                    ) {
    return new StepBuilder(name:"consolidacionDiariaStep", jobRepository) // Crea un Step llamado "consolidacionDiariaStep"
            .<Venta, InformeVenta>chunk(new SimpleCompletionPolicy(chunkSize:2), transactionManager) // Define el tamaño del chunk y la política de trans
            .reader(itemReader) // Configura el lector de ítems
            .processor(itemProcessor) // Configura el procesador de ítems
            .writer(itemWriter) // Configura el escritor de ítems
            .faultTolerant() // Habilita la tolerancia a fallos en el Step
            .skipPolicy(fileVerificationSkipper) // Asigna una política de salto personalizada
            .listener(skipListener)  // Añade un SkipListener para manejar errores
            .listener(errorFileStepExecutionListener) // Añade un listener para ejecutar acciones específicas en el Step
            .taskExecutor(taskExecutor) // Configura la ejecución en paralelo con un TaskExecutor
            .build(); // Construye y devuelve el Step configurado
}
```

Nota. Estructura avanzada de un step; se observa un Step configurado para procesamiento paralelo, reintentos y política de repetición a través de Power Shell. *Microsoft (2006).* Power Shell (7.4.5). [Software]. https://learn.microsoft.com/en-us/powershell/

Por último, se utiliza un TaskExecutor para optimizar la ejecución en paralelo, permitiendo que los chunks se procesen simultáneamente, lo que mejora la eficiencia del procesamiento en lotes. Esta configuración asegura que el flujo del trabajo sea robusto y eficiente, incluso ante fallos o datos inconsistentes.

En la Figura 6 se muestra una configuración importante porque define el TaskExecutor, que se encargará de gestionar la ejecución en paralelo dentro de tu proceso batch. Al utilizar un 'ThreadPoolTaskExecutor', puedes controlar el número de hilos que se usan simultáneamente, optimizando así el uso de recursos del sistema.

Figura 6

Componente de optimización de ejecución

```
// Configura el TaskExecutor para procesamiento paralelo
@Bean
public ThreadPoolTaskExecutor taskExecutor() {
    ThreadPoolTaskExecutor executor = new ThreadPoolTaskExecutor(); // Crea un TaskExecutor para la ejecución en paralelo
    executor.setCorePoolSize(corePoolSize:5); // Define el tamaño inicial del pool de hilos
    executor.setMaxPoolSize(maxPoolSize:10); // Define el tamaño máximo del pool de hilos
    executor.setQueueCapacity(queueCapacity:25); // Define la capacidad de la cola
    executor.setThreadNamePrefix(threadNamePrefix:"Batch-Thread-"); // Prefijo de nombre de hilo para identificar los hilos del batch
    executor.initialize(); // Inicializa el TaskExecutor
    return executor;
}
```

Nota. Configuración de un step para considerar políticas de optimización; configura el TaskExecutor para procesamiento paralelo, a través de Power Shell. *Microsoft (2006).* Power Shell. (7.4.5). [Software]. https://learn.microsoft.com/en-us/powershell/

Revisemos los parámetros con mayor detalle:

- **setCorePoolSize**(5): este parámetro establece el tamaño mínimo del pool de hilos, lo que significa que siempre habrá al menos 5 hilos disponibles para procesar tareas. Esto es útil si necesitas un procesamiento constante y quieres evitar que los hilos se creen y destruyan repetidamente.
- **setMaxPoolSize**(10): aquí defines el tamaño máximo de hilos en el pool. En momentos de alta demanda, el TaskExecutor puede crear hasta 10 hilos para procesar tareas en paralelo, lo que mejora el rendimiento cuando tienes una gran cantidad de tareas por procesar.
- **setQueueCapacity**(25): esta configuración define la capacidad de la cola de espera para tareas cuando todos los hilos del pool están ocupados. Con una cola de 25, puedes manejar picos de trabajo sin perder tareas, ya que las tareas esperarán en la cola hasta que un hilo esté disponible.

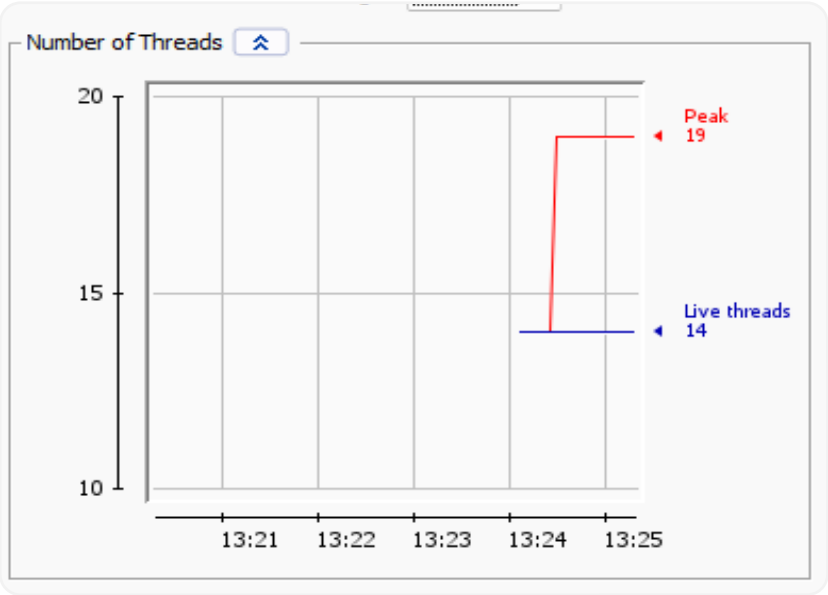
- **setThreadNamePrefix**("Batch-Thread-"): este prefijo facilita la identificación de los hilos en los logs, ya que todos los hilos que gestione este TaskExecutor comenzarán con "Batch-Thread-". Esto es útil para el monitoreo y la depuración.
- **initialize**(): con esta llamada, inicializas el TaskExecutor para que esté listo para gestionar las tareas.

Esta configuración te permite controlar de forma efectiva cómo y cuándo se ejecutan las tareas en paralelo, mejorando el rendimiento y facilitando el monitoreo y la gestión de recursos en tu aplicación batch.

Si analizas el proceso Java que se ejecuta en el sistema operativo, podrás ver como el step crea los 5 hilos nuevos que fueron definidos en el TaskExecutor. Puedes observar un gráfico de rendimiento en las figuras 7 (hilos) y 8 (cpu).

Figura 7

Monitoreo de hilos en el sistema operativo

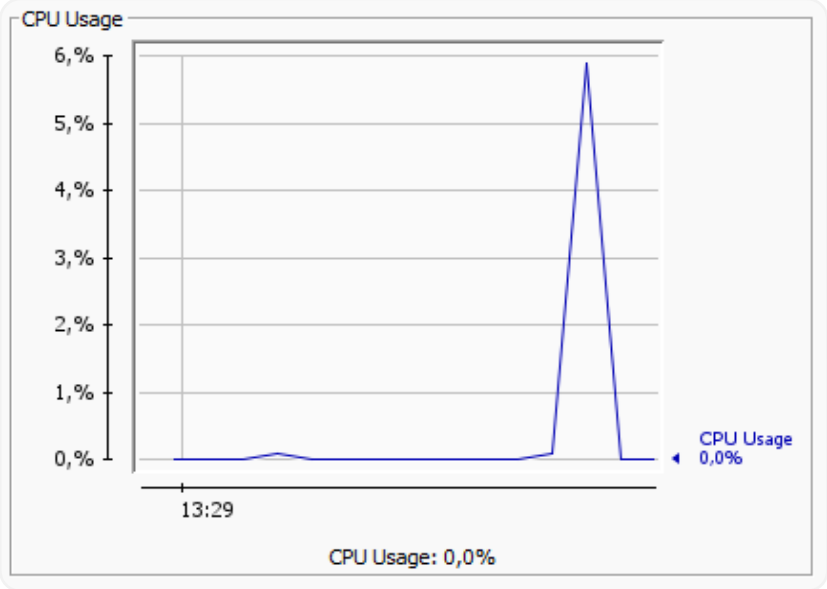


Nota. Captura que muestra el aumento de hilos en ejecución a nivel de sistema operativo.

Oracle. (2024). JDK 23. (23.0.1). [Software]
https://www.oracle.com/cl/java/technologies/downloads/

Figura 8

Uso de CPU en el sistema operativo



Nota. Captura que muestra el aumento de CPU cuando el batch es ejecutado. *Oracle. (2024).* JDK 23. (23.0.1). [Software] https://www.oracle.com/cl/java/technologies/downloads/

Finalmente, cuando ejecutes el proceso obtendrás una salida similar a la que está a continuación:

```
[main] INFO org.springframework.batch.core.configuration.annotation.BatchRegistrar - Finished Spring Batch infrastructure beans configuration in 18 ms.
[main] WARN org.springframework.context.support.PostProcessorRegistrationDelegate$BeanPostProcessorChecker - Bean 'jobRegistry' of type [org.springframework.batch.core.configuration.support.MapJobRegistry] is not eligible for getting processed by all BeanPostProcessors (for example: not eligible for auto-proxying). Is this bean getting eagerly injected into a currently created BeanPostProcessor [jobRegistryBeanPostProcessor]? Check the corresponding BeanPostProcessor declaration and its dependencies.
[main] INFO com.duoc.advanced.ErrorFileStepExecutionListener - Se ejecuto el ErrorFileStepExecutionListener
[main] INFO org.springframework.jdbc.datasource.embedded.EmbeddedDatabaseFactory - Starting embedded database: url='jdbc:h2:mem:testdb;DB_CLOSE_DELAY=-1;DB_CLOSE_ON_EXIT=false', username='sa'
[main] INFO org.springframework.batch.core.repository.support.JobRepositoryFactoryBean - No database type set, using meta data indicating: H2
[main] INFO com.duoc.advanced.VentaSkipListener - Creando VentaSkipListener
[main] WARN org.springframework.batch.core.step.builder.FaultTolerantStepBuilder - Asynchronous TaskExecutor detected with ItemStream reader.  This is probably an error, and may lead to incorrect restart data being stored.
[main] INFO org.springframework.batch.core.configuration.annotation.BatchObservabilityBeanPostProcessor - No Micrometer observation registry found, defaulting to ObservationRegistry.NOOP
```

```
[main] INFO org.springframework.batch.core.configuration.annotation.BatchObservabilityBeanPostProcessor - No Micrometer observation registry found, defaulting to ObservationRegistry.NOOP
[Batch-Thread-1] INFO org.springframework.batch.core.launch.support.SimpleJobLauncher - Job: [FlowJob: [name=ventasJob]] launched with the following parameters: [{'time':'{value=1730392540042, type=class java.lang.Long, identifying=true}'}]
[Batch-Thread-1] INFO com.duoc.advanced.JobCompletionListener - Inicio del trabajo batch con ID: 1
[Batch-Thread-1] INFO org.springframework.batch.core.job.SimpleStepHandler - Executing step: [consolidacionDiariaStep]
[Batch-Thread-1] INFO com.duoc.advanced.ErrorFileStepExecutionListener - Abre el archivo de errores al inicio
[Batch-Thread-1] WARN org.springframework.batch.core.step.item.ChunkMonitor - No ItemReader set (must be concurrent step), so ignoring offset data.
[Batch-Thread-3] INFO com.duoc.advanced.FileVerificationSkipper - Error: FlatFileParseException
[Batch-Thread-2] INFO com.duoc.advanced.FileVerificationSkipper - Error: FlatFileParseException
[Batch-Thread-4] INFO com.duoc.advanced.FileVerificationSkipper - Error: FlatFileParseException
[Batch-Thread-2] WARN com.duoc.advanced.FileVerificationSkipper - CustomSkipPolicy - Excepción omitida: Parsing error at line: 7 in resource=[class path resource [consolidacion_diaria_ventas.csv]], input=[6,Producto F,10          ]
[Batch-Thread-3] WARN com.duoc.advanced.FileVerificationSkipper - CustomSkipPolicy - Excepción omitida: Parsing error at line: 5 in resource=[class path resource [consolidacion_diaria_ventas.csv]], input=[3,Producto C,10,abc       ]
[Batch-Thread-2] WARN com.duoc.items.VentasItemProcessor - Cantidad inválida para producto Producto G: -5
[Batch-Thread-5] WARN org.springframework.batch.core.step.item.ChunkMonitor - ItemStream was opened in a different thread. Restart data could be compromised.
[Batch-Thread-4] WARN com.duoc.advanced.FileVerificationSkipper - CustomSkipPolicy - Excepción omitida: Parsing error at line: 5 in resource=[class path resource [consolidacion_diaria_ventas.csv]], input=[2,Producto B, ,200.0      ]
[Batch-Thread-3] WARN com.duoc.items.VentasItemProcessor - Precio inválido para producto Producto H: null
[Batch-Thread-4] INFO com.duoc.advanced.FileVerificationSkipper - Error: FlatFileParseException
[Batch-Thread-4] WARN com.duoc.advanced.FileVerificationSkipper - CustomSkipPolicy - Excepción omitida: Parsing error at line: 11 in resource=[class path resource [consolidacion_diaria_ventas.csv]], input=[10,Producto J,5,texto      ]
[Batch-Thread-3] INFO com.duoc.advanced.FileVerificationSkipper - Error: FlatFileParseException
[Batch-Thread-4] INFO com.duoc.advanced.VentaSkipListener - Entro a onSkipInRead: org.springframework.batch.item.file.FlatFileParseException
[Batch-Thread-4] WARN com.duoc.advanced.VentaSkipListener - Línea omitida debido a un error en la lectura: 2,Producto B, ,200.0
[Batch-Thread-4] INFO com.duoc.advanced.VentaSkipListener - Entro a onSkipInRead: org.springframework.batch.item.file.FlatFileParseException
[Batch-Thread-4] WARN com.duoc.advanced.VentaSkipListener - Línea omitida debido a un error en la lectura: 10,Producto J,5,texto
[Batch-Thread-4] WARN org.springframework.batch.core.step.item.ChunkMonitor - ItemStream was opened in a different thread. Restart data could be compromised.
[Batch-Thread-3] WARN com.duoc.advanced.FileVerificationSkipper - CustomSkipPolicy - Excepción omitida: Cantidad debe ser mayor que 0
[Batch-Thread-3] WARN org.springframework.batch.core.step.item.ChunkMonitor - ItemStream was opened in a different thread. Restart data could be compromised.
[Batch-Thread-3] INFO com.duoc.advanced.VentaSkipListener - Tipo de excepción en SkipListener: com.duoc.business.InvalidDataException
[Batch-Thread-3] WARN com.duoc.advanced.VentaSkipListener - SkipListener activado - Error al procesar registro: com.duoc.business.Venta@365d33ec
[Batch-Thread-3] INFO com.duoc.advanced.VentaSkipListener - Registro escrito en errores.csv: com.duoc.business.Venta@365d33ec
```

```
[Batch-Thread-3] INFO com.duoc.advanced.VentaSkipListener - Entro a onSkipInRead:
org.springframework.batch.item.file.FlatFileParseException
[Batch-Thread-3] WARN com.duoc.advanced.VentaSkipListener - Línea omitida debido a un error en la lectura: 6,Producto F,10

 [Batch-Thread-2] INFO com.duoc.advanced.FileVerificationSkipper - Error: FlatFileParseException
[Batch-Thread-2] WARN com.duoc.advanced.FileVerificationSkipper - CustomSkipPolicy - Excepción omitida: Precio debe ser mayor
que 0
[Batch-Thread-2] WARN org.springframework.batch.core.step.item.ChunkMonitor - ItemStream was opened in a different thread.
Restart data could be compromised.
[Batch-Thread-2] INFO com.duoc.advanced.VentaSkipListener - Tipo de excepción en SkipListener:
com.duoc.business.InvalidDataException
[Batch-Thread-2] WARN com.duoc.advanced.VentaSkipListener - SkipListener activado - Error al procesar registro:
com.duoc.business.Venta@260288c8
[Batch-Thread-2] INFO com.duoc.advanced.VentaSkipListener - Registro escrito en errores.csv:
com.duoc.business.Venta@260288c8
[Batch-Thread-2] INFO com.duoc.advanced.VentaSkipListener - Entro a onSkipInRead:
org.springframework.batch.item.file.FlatFileParseException
[Batch-Thread-2] WARN com.duoc.advanced.VentaSkipListener - Línea omitida debido a un error en la lectura: 3,Producto C,10,abc
[Batch-Thread-1] INFO com.duoc.advanced.ErrorFileStepExecutionListener - Cierra el archivo de errores al final
[Batch-Thread-1] INFO org.springframework.batch.core.step.AbstractStep - Step: [consolidacionDiariaStep] executed in 257ms
[Batch-Thread-1] INFO com.duoc.advanced.JobCompletionListener - Job completado exitosamente con ID: 1
[Batch-Thread-1] INFO com.duoc.advanced.JobCompletionListener - Resumen del Job: JobExecution: id=1, version=1,
startTime=2024-10-31T13:35:40.182368300, endTime=2024-10-31T13:35:40.512002700, lastUpdated=2024-10-
31T13:35:40.183368300,
status=COMPLETED, exitStatus=exitCode=COMPLETED;exitDescription=, job=[JobInstance: id=1, version=0, Job=[ventasJob]],
jobParameters=[{'time':'{value=1730392540042, type=class java.lang.Long, identifying=true}'}]
[Batch-Thread-1] INFO org.springframework.batch.core.launch.support.SimpleJobLauncher - Job: [FlowJob: [name=ventasJob]]
completed with the following parameters: [{'time':'{value=1730392540042, type=class java.lang.Long, identifying=true}'}] and the
following status: [COMPLETED] in 329ms
[main] INFO org.springframework.jdbc.datasource.embedded.EmbeddedDatabaseFactory - Shutting down embedded database:
url='jdbc:h2:mem:testdb;DB_CLOSE_DELAY=-1;DB_CLOSE_ON_EXIT=false'
```

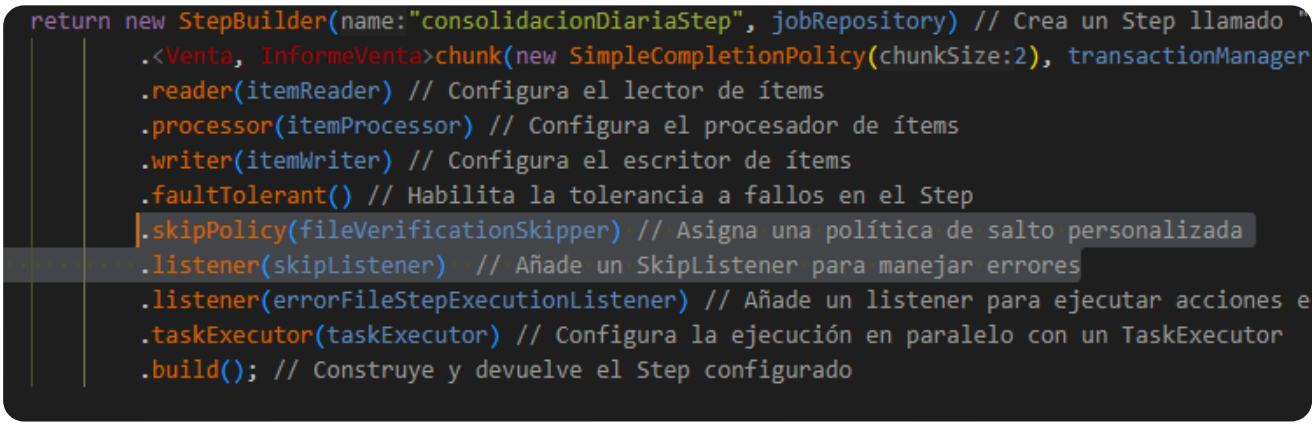
Los Logs destacados, búscalos en el código, ya que estos son provocados por la implementación. La idea principal tras el hecho de gestionar correctamente logs en consola es poder “trazar” o “depurar” la ejecución, y además confirmar lo observado a nivel de sistema operativo.

Manejo de errores

Para poder manejar errores en un proceso batch, debes considerar dos partes de este:

Figura 9

Manejo de errores



Nota. Captura de pantalla que muestra los dos componentes (fileVerificationSkipper y skipListener) de un manejo de errores, a través de Power Shell. *Microsoft (2006).* Power Shell. (7.4.5). [Software]. https://learn.microsoft.com/en-us/powershell/

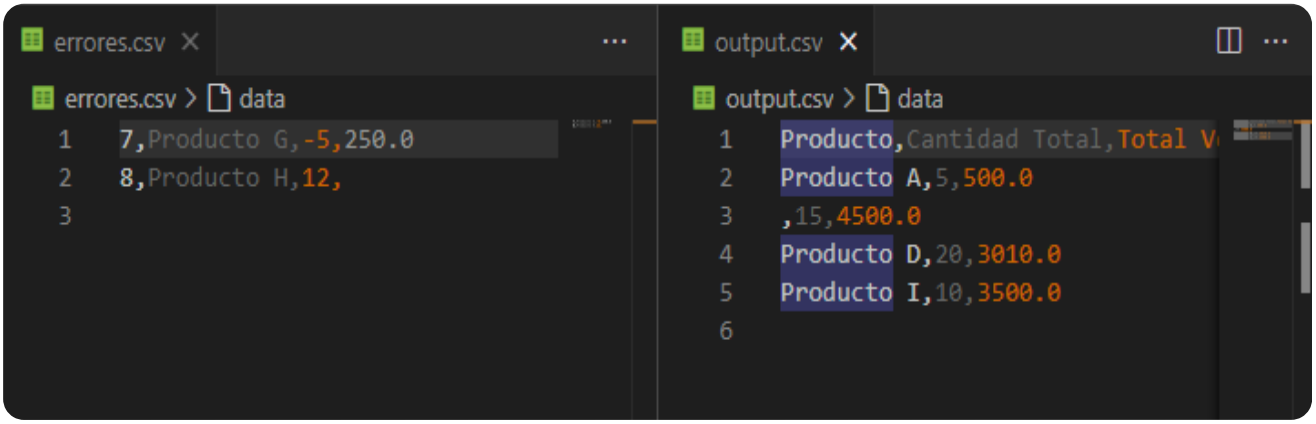
El manejo de errores en esta configuración ocurre gracias a:

- **Política de Salto (SkipPolicy):** permite que el Step ignore errores específicos y continúe procesando los elementos restantes. En la implementación InvalidDataException y FlatFileParseException son las excepciones especificas manejadas por el proceso.
- **SkipListener:** registra los elementos omitidos y los errores, proporcionando información útil para identificar problemas sin detener el flujo.
- **ErrorFileStepExecutionListener:** permite realizar acciones adicionales relacionadas con el manejo de errores, como registrar todos los errores en un archivo, lo cual es útil para revisar después del procesamiento.

Resultado final

Figura 10

Resultado del proceso



Nota. Imagen que muestra la salida final del batch, incluyendo un archivo de errores (errores.csv) y un archivo de salida (output.csv). *Microsoft. (2024).* Visual Studio Code. (1.95). [Software]. https://code.visualstudio.com/

Links de Interés

A continuación, podrás revisar ejemplos de casos de uso Spring Batch:
https://github.com/spring-projects/spring-batch/tree/main/spring-batch-samples

Cierre de la semana



Al cerrar esta semana, has fortalecido tus habilidades en la configuración y optimización de procesos batch en Spring Batch, utilizando técnicas avanzadas para mejorar la eficiencia y estabilidad del sistema. Ahora eres capaz de implementar estrategias de escalado, como el procesamiento en chunks, el paralelismo con múltiples hilos, para manejar grandes volúmenes de información de manera más rápida y eficaz. Estas prácticas te permiten distribuir la carga de trabajo y aprovechar mejor los recursos del sistema, reduciendo los tiempos de ejecución y mejorando el rendimiento general.

Además, has aprendido a configurar políticas de finalización y re-ejecución para gestionar la continuidad de los procesos batch. Ahora puedes determinar cuándo es necesario reiniciar un job, omitir errores o reintentar operaciones, garantizando así que el sistema sea más resiliente frente a fallos y pueda recuperar su flujo de trabajo sin afectar la integridad de los datos. Estas políticas son fundamentales para asegurar que los trabajos batch se completen con éxito incluso en condiciones adversas.

Finalmente, comprendiste la importancia de optimizar los recursos del sistema y el manejo de errores. Al aplicar técnicas para limitar el uso de memoria, ajustar el tamaño de los chunks y monitorizar el rendimiento en tiempo real, te aseguras de que los procesos batch sean lo más eficientes y estables posible. El manejo adecuado de errores y excepciones también garantiza que el sistema siga funcionando de manera fluida, minimizando interrupciones y maximizando la confiabilidad del entorno batch.

Referencias

- Spring Batch (s.f.). *Spring batch architecture.* Spring. https://docs.spring.io/spring-batch/reference/spring-batch-architecture.html
- Spring Batch (s.f.). *Chunk-oriented Processing.* Spring. https://docs.spring.io/spring-batch/reference/step/chunk-oriented-processing.html

Lecturas de la semana

- Spring (2024). Scaling and Parallel Processing https://docs.spring.io/spring-batch/reference/scalability.html
- Spring. (2024). *Spring Batch Architecture.* https://docs.spring.io/spring-batch/reference/spring-batch-architecture.html

Apuntes



Reservados todos los derechos Fundación Instituto Profesional Duoc UC. No se permite copiar, reproducir, reeditar, descargar, publicar, emitir, difundir, de forma total o parcial la presente obra, ni su incorporación a un sistema informático, ni su transmisión en cualquier forma o por cualquier medio (electrónico, mecánico, fotocopia, grabación u otros) sin autorización previa y por escrito de Fundación Instituto Profesional Duoc UC
La infracción de dichos derechos puede constituir un delito contra la propiedad intelectual.